



北京航空航天大学
BEIHANG UNIVERSITY

机器人导航python实践(入门)

Lecture7-路径规划 part2

北航 国新院 实验实践课
智能系统与人形机器人国际研究中心



教师： 欧 阳 老 师



邮 箱： ouyangkid@buaa.edu.cn



学 期： 2025年秋季

目录

Contents

01 课程内容安排

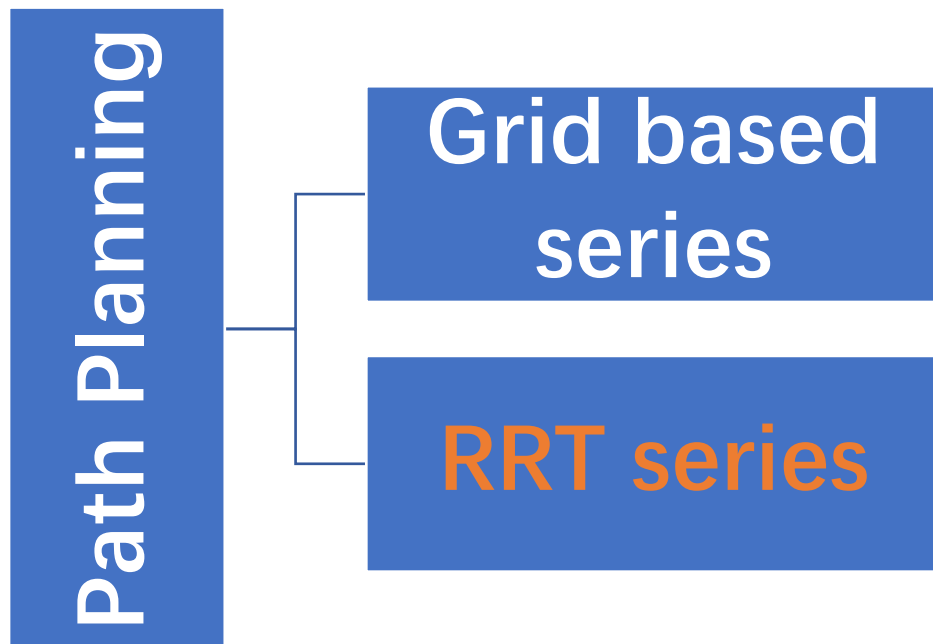
02 Grid based Search

03 RRT

04

05

06



路径规划 (Path Planning) 指机器人搜索前往目标点的可行且高效路径的能力。该路径需依据机器人的运动模型和障碍物位置满足特定约束条件，并对部分目标函数进行优化，例如到达目标点的时间、与障碍物的距离等。

基于网格地图的算法，路径搜索和规划的空间通常被定义为2D数据结构；

而**搜索树方法**则利用树结构提升表达和计算的效率。

目录

Contents

01 RRT

02 RRT with path smoothing

03 RRT with sobol sampler

04 RRT star

05 Informed RRT star

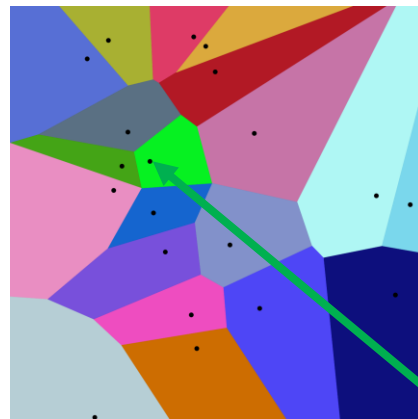
Part 1 | 随机树快速搜索

» RRT

RRT通过随机构建空间填充树实现对非凸高维空间的搜索，通过随机抽取样本，并倾向于向空间中未被大量搜索的区域生长，逐步构建。能够轻松处理带有障碍和微分约束（如运动学约束、空间约束、速度加速度等动力学）的问题。

快速探索随机树（RRT）通过从搜索空间中随机采样，生成一棵以**起始配置为根**的树。每次采样后，都会尝试将该样本与其在树中最近的状态建立连接。如果连接可行（完全通过自由空间且遵守所有约束），则会将新状态添加到树中。在对搜索空间进行均匀采样时，扩展现有状态的概率与其Voronoi区域（**泰森多边形**）的大小成正比。由于最大的Voronoi区域属于搜索边界上的状态，这意味着树会优先向未搜索的大面积区域扩展。

泰森多边形在空间剖分上的等分性特征，可用离散点的数据来计算泰森多边形区域的数据；判断一个离散点与其它哪些离散点相邻时，可根据泰森多边形直接得出。



绿色区域内的点，到自己种子点的距离，比到其他种子点距离更近。

树与新状态之间的连接长度通常受生长因子限制。如果随机样本与其在树中最近的状态之间的距离超过了该限制所允许的范围，那么将使用沿指向随机样本的直线上距离树最远（达到最大距离）的新状态，而非随机样本本身。此时，随机样本可被视为控制树的生长方向，而生长因子则决定生长速度。这在限制增量生长规模的同时，保留了快速探索随机树（RRT）的空间填充倾向。

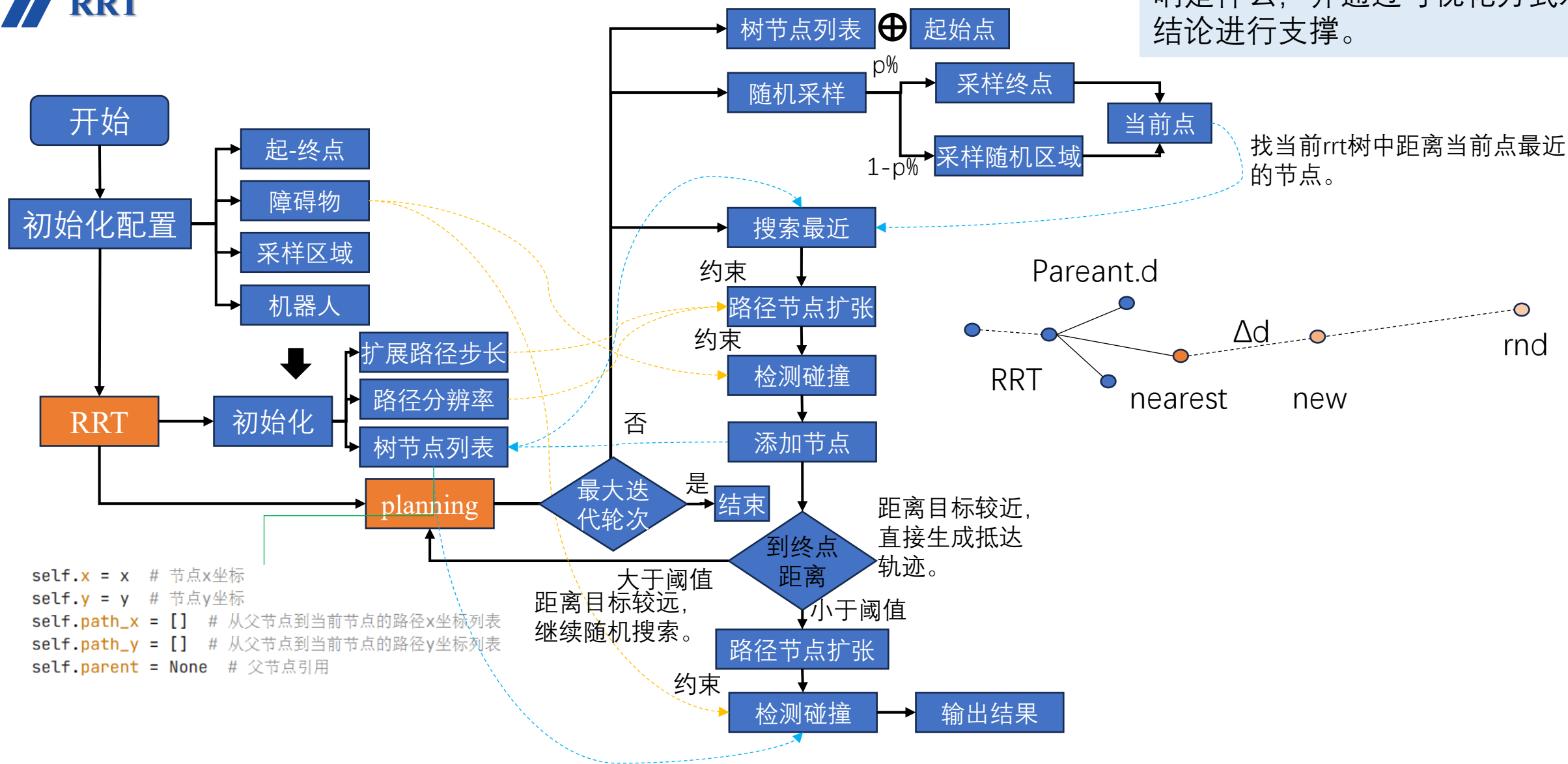
无法理解？不要紧，
算法**隐式逼近**该理论



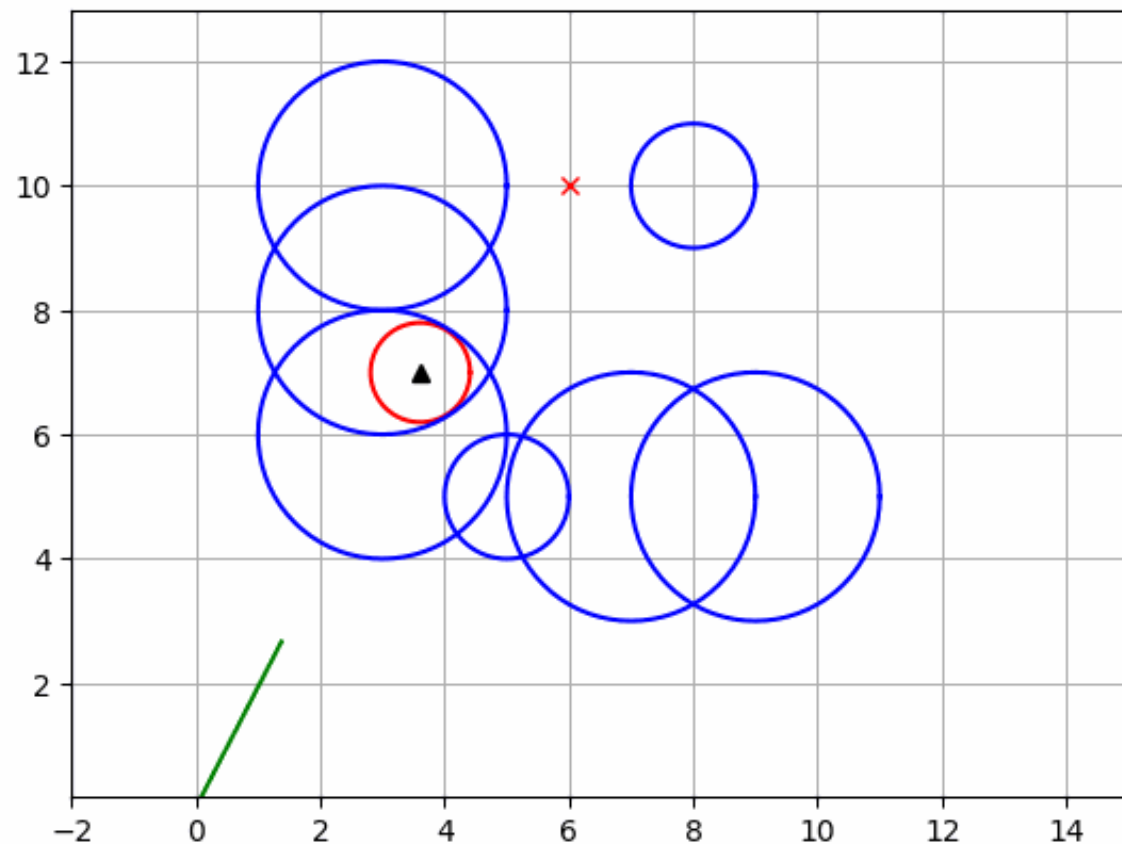
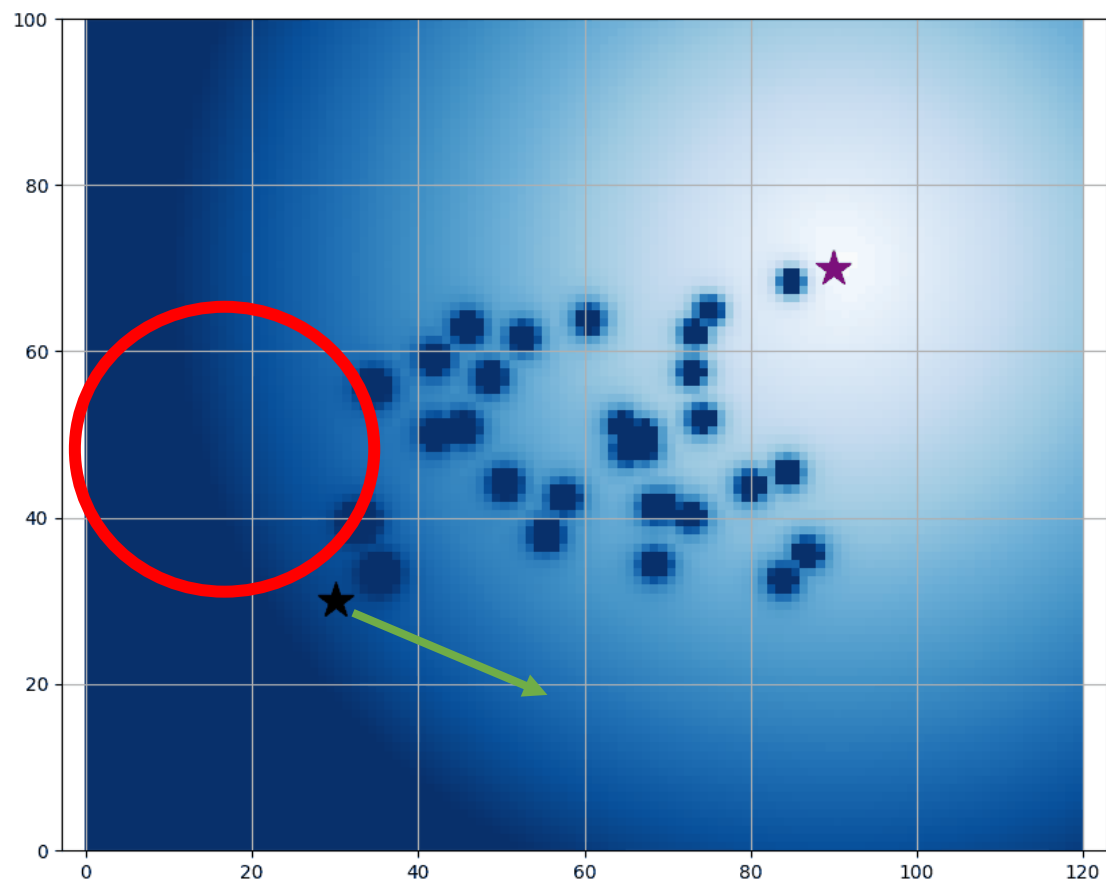
Part 1 | 随机树快速搜索

» RRT

实验要求：调整不同的 p 值（如5%），评估RRT搜索到最终路径的平均探索步长，分析 p 对算法的影响是什么，并通过可视化方式对结论进行支撑。



Part 1 | RRT



势场导航中，一旦初始（运动模型指向上）
选择了一个搜索方向，即便在中间会因为绕
路，也很难尝试从右下方进行扩展的路径。

目录

Contents

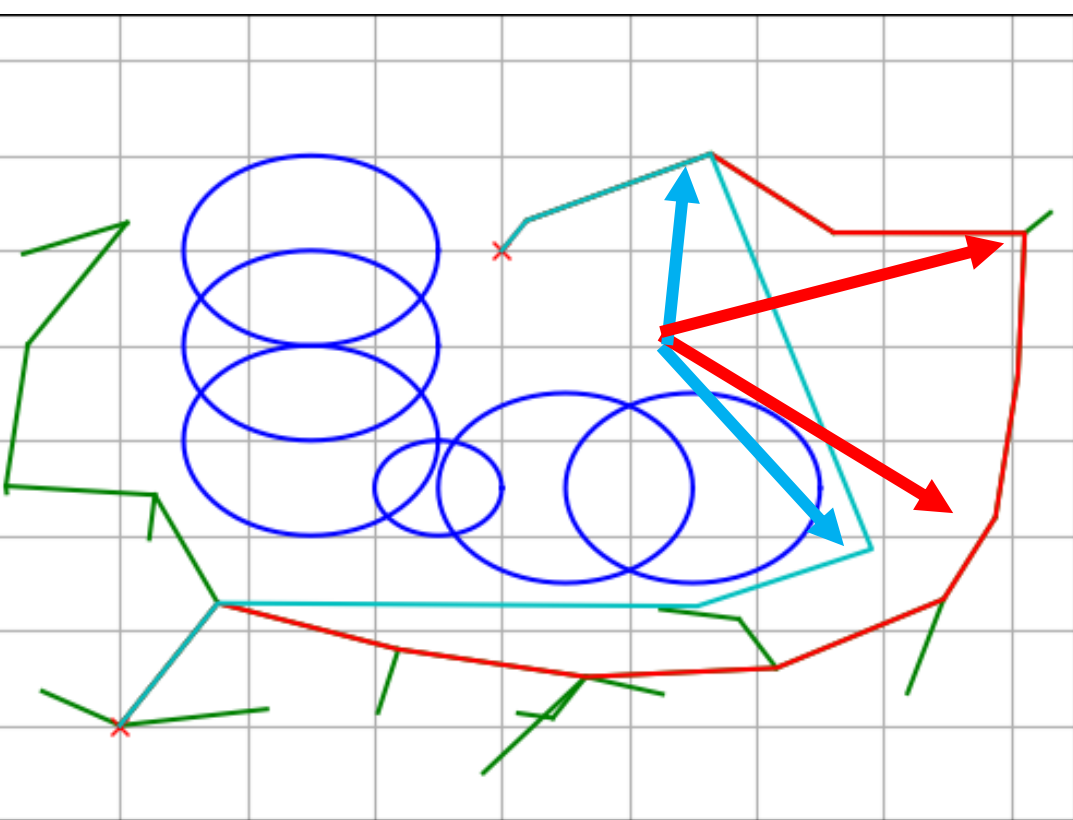
01 RRT

02 RRT with path smoothing

03 RRT with sobol sampler

04 RRT star

05 Informed RRT star



RRT

初始化

路径分辨率

树节点列表

planning

```
self.x = x # 节点x坐标
self.y = y # 节点y坐标
self.path_x = [] # 从父节点到当前节点的路径x坐标列表
self.path_y = [] # 从父节点到当前节点的路径y坐标列表
self.parent = None # 父节点引用
```

大于阈值
距离目标较远，
继续随机搜索。

树节点列表

随机采样

搜索最近

约束

路径节点扩张

约束

检测碰撞

添加节点

距离目标较近，直接生成抵达轨迹。

到终点
距离

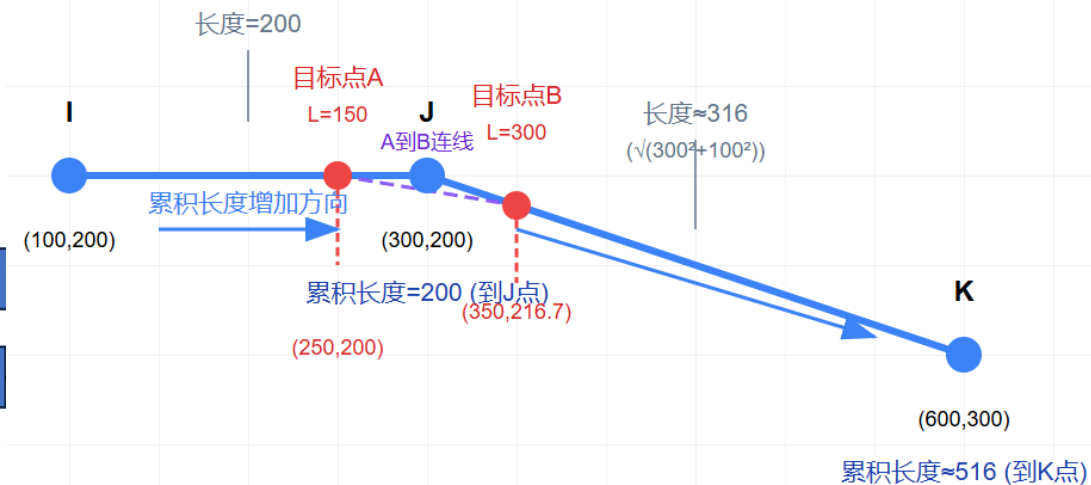
小于阈值

路径节点扩张

约束

检测碰撞

平滑路径



随机选取不相邻点，进行直连，替代原路径的同时，避免与障碍我碰撞。

计算路径长度le:累加

迭代x轮

迭代随机选点: [0, le]

选择两个点AB, 排序

A在原路径的坐标和索引

B在原路径的坐标和索引

A、B索引不越界

机器人在A-B不碰撞

A-B替代原折线段

目录

Contents

01 RRT

02 RRT with path smoothing

03 RRT with sobol sampler

04 RRT star

05 Informed RRT star

Part 3 | Sobol sampler

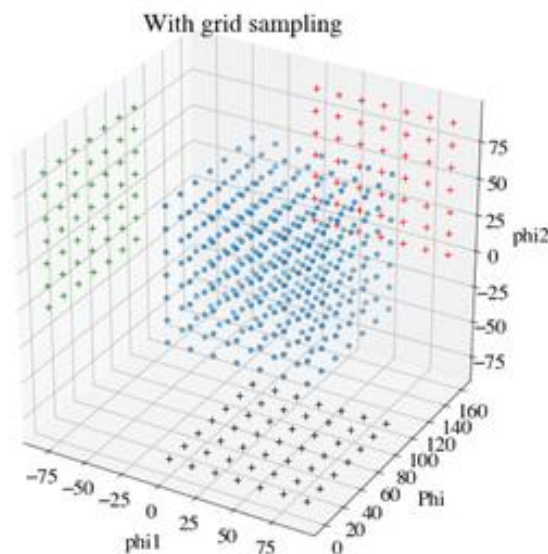
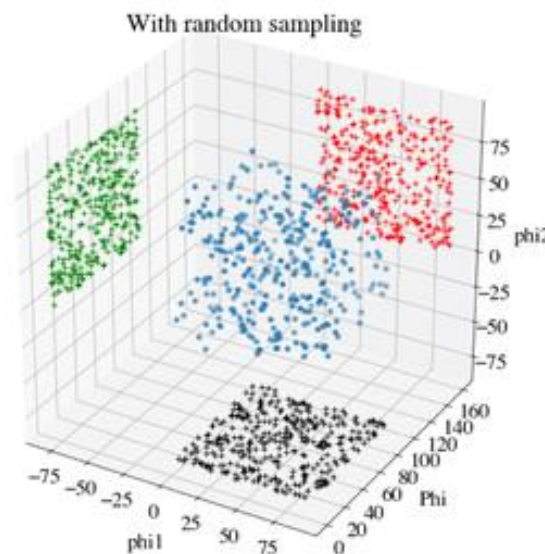
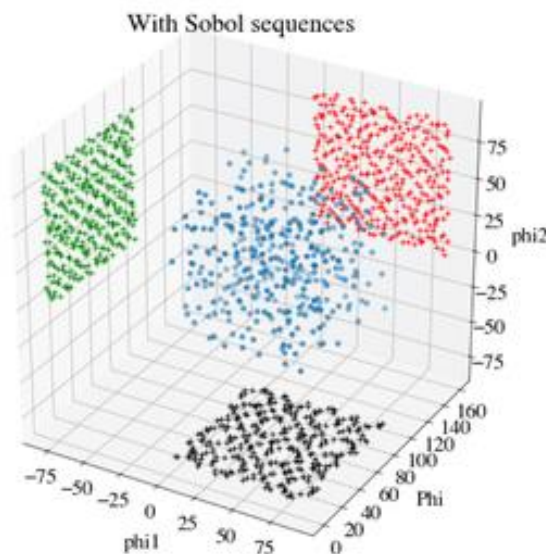
随机采样

Sobol sequence 索博尔序列，以2为基数，对单位区间进行连续更精细的均匀划分，然后对每个维度的坐标进行重新排序。

对应的sobol采样，是一种基于**Sobol 序列**的低差异采样方法，用于生成探索空间中的采样点。与完全随机采样相比，它能生成**分布更均匀**的点集，从而更高效地覆盖探索空间，理论上可提升路径搜索的效率。

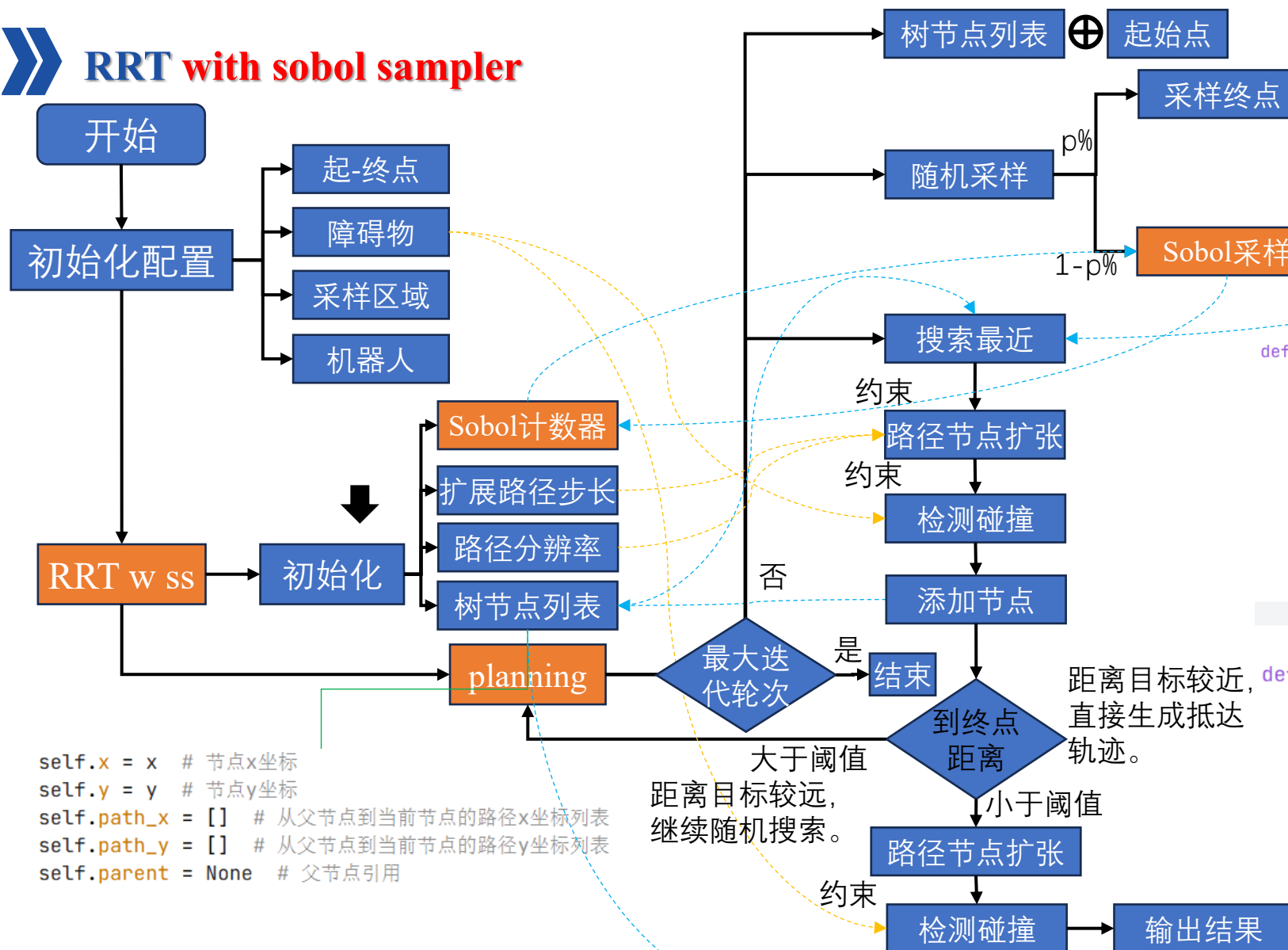
随机，但不均匀

均匀，但缺乏随机性



Part 3 | 随机树快速搜索

» RRT with sobol sampler



实验要求：后两种关于RRT的改进主要针对更加庞大的地图和复杂的路径。扩大场景，并将障碍物设置为随机，在进行对比测试，并对整个过程进行可视化输出。

```
def get_random_node(self):
    if random.randint(0, 100) > self.goal_sample_rate:
        rand_coordinates, n = sobol_quasirand(2, self.sobol_inter_)

        rand_coordinates = self.min_rand + \
            rand_coordinates*(self.max_rand - self.min_rand)
        self.sobol_inter_ = n
        rnd = self.Node(*rand_coordinates)
```

```
else: # goal point sampling
    rnd = self.Node(self.end.x, self.end.y)
return rnd
```

```
def get_random_node(self):
    if random.randint(0, 100) > self.goal_sample_rate:
        rnd = self.Node(
            random.uniform(self.min_rand, self.max_rand),
            random.uniform(self.min_rand, self.max_rand))
    else: # goal point sampling
        rnd = self.Node(self.end.x, self.end.y)
    return rnd
```

```
self.x = x # 节点x坐标
self.y = y # 节点y坐标
self.path_x = [] # 从父节点到当前节点的路径x坐标列表
self.path_y = [] # 从父节点到当前节点的路径y坐标列表
self.parent = None # 父节点引用
```

目录

Contents

01 RRT

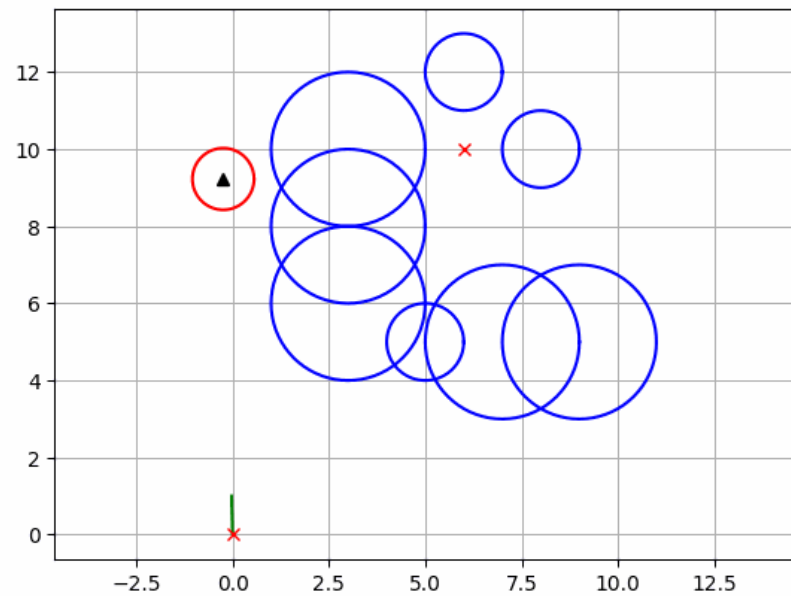
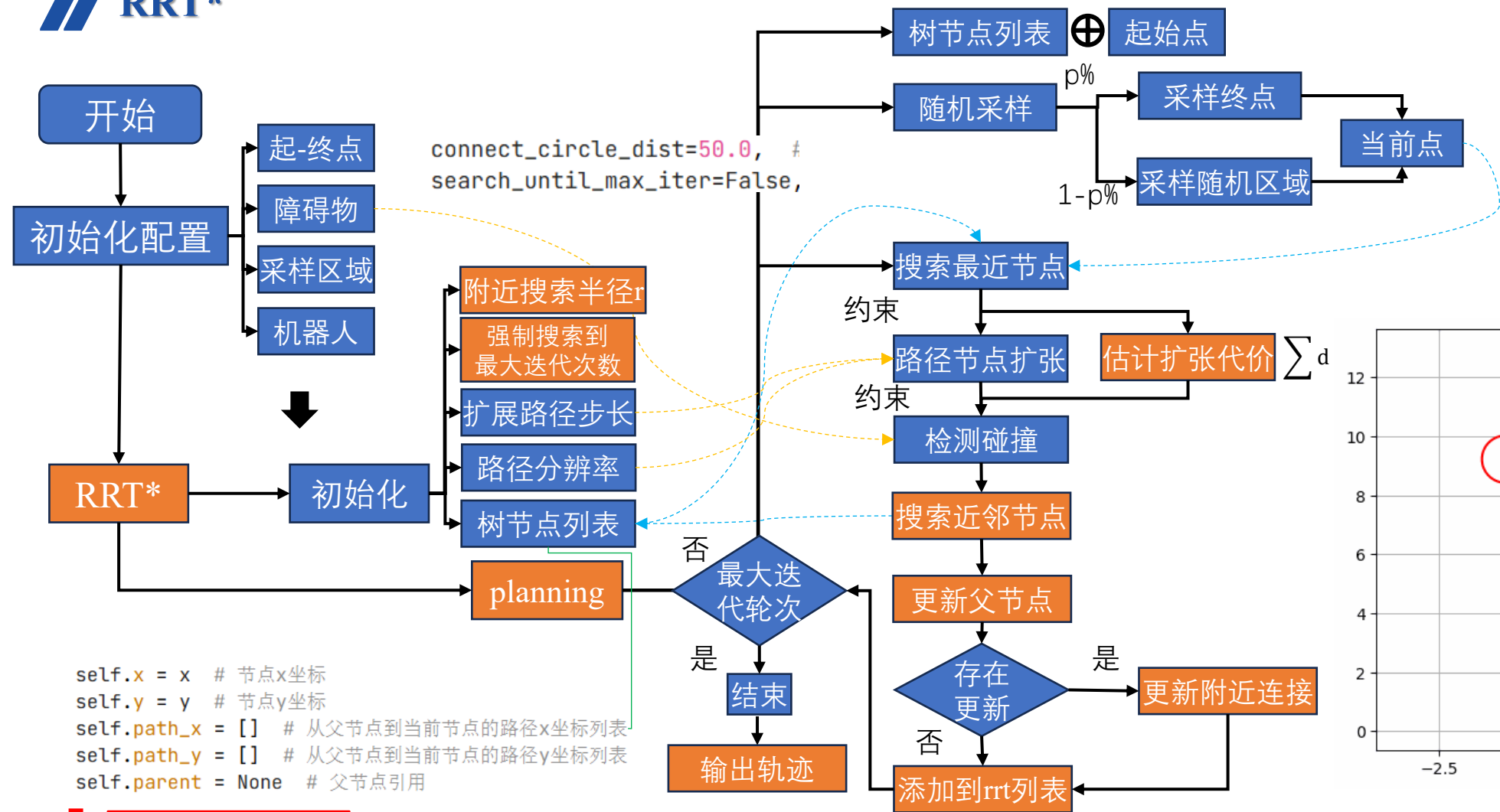
02 RRT with path smoothing

03 RRT with sobol sampler

04 RRT star

05 Informed RRT star

Part 4 | 随机树快速搜索 star



+ `self.cost = 0.0`

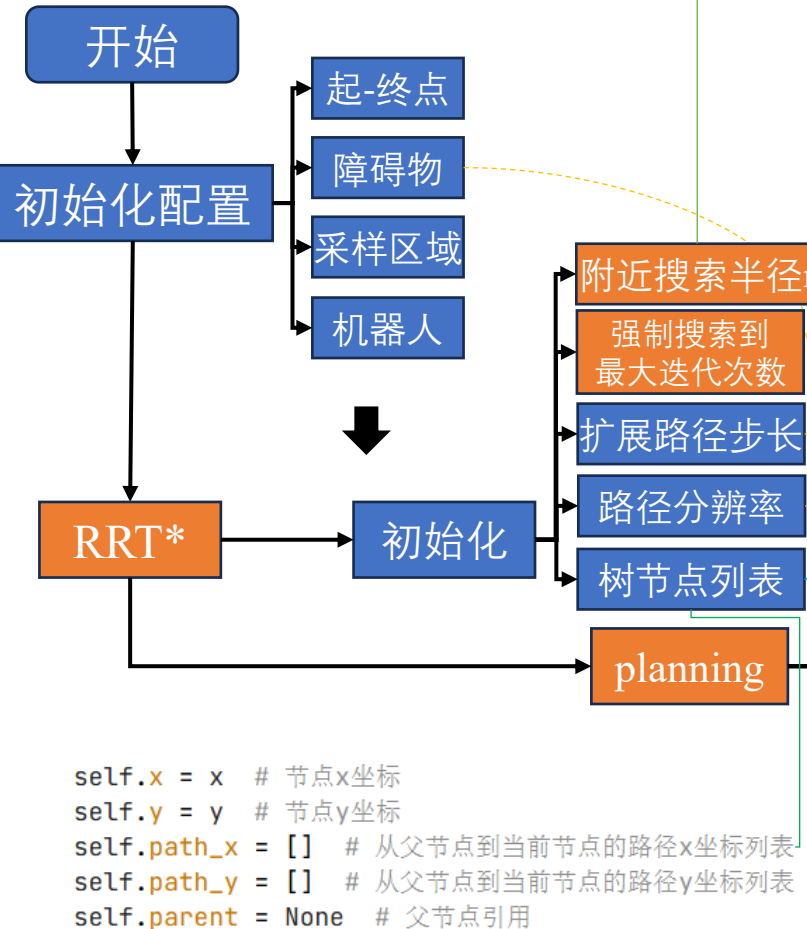
Part 4 | 随机树快速搜索 star

存在一个动态调整的策略



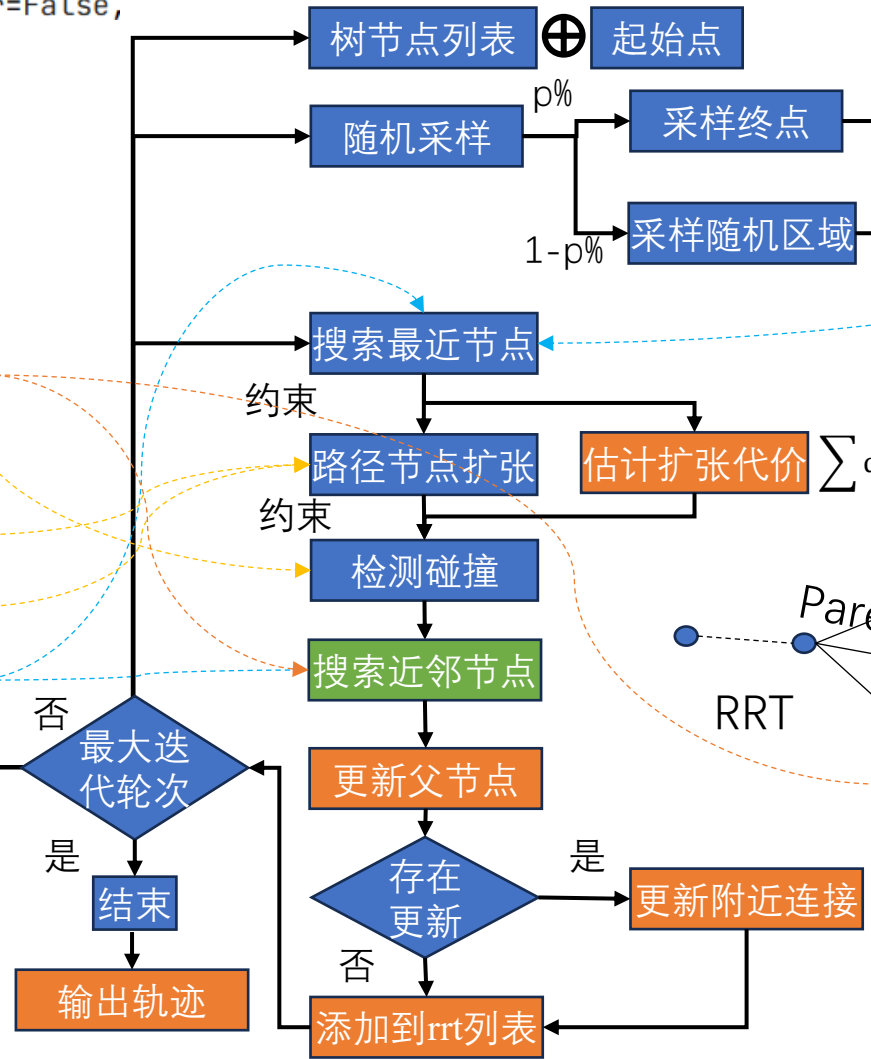
```
connect_circle_dist=50.0, #
search_until_max_iter=False,
```

```
r = self.connect_circle_dist * math.sqrt(math.log(nnode) / nnode)
```

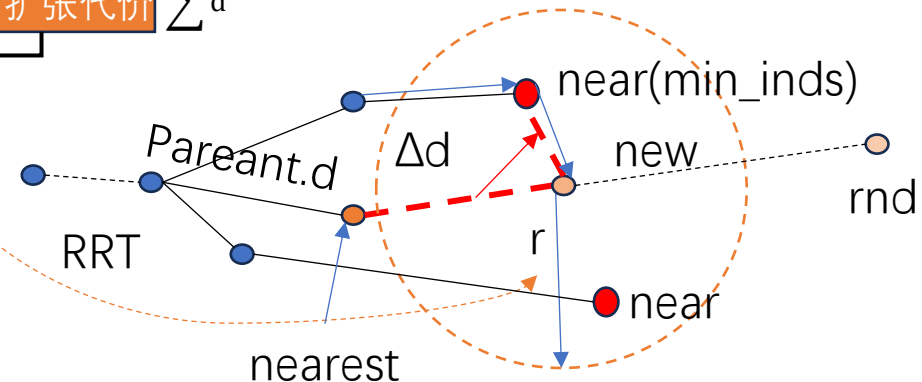


```
self.x = x # 节点x坐标
self.y = y # 节点y坐标
self.path_x = [] # 从父节点到当前节点的路径x坐标列表
self.path_y = [] # 从父节点到当前节点的路径y坐标列表
self.parent = None # 父节点引用
```

+ self.cost = 0.0

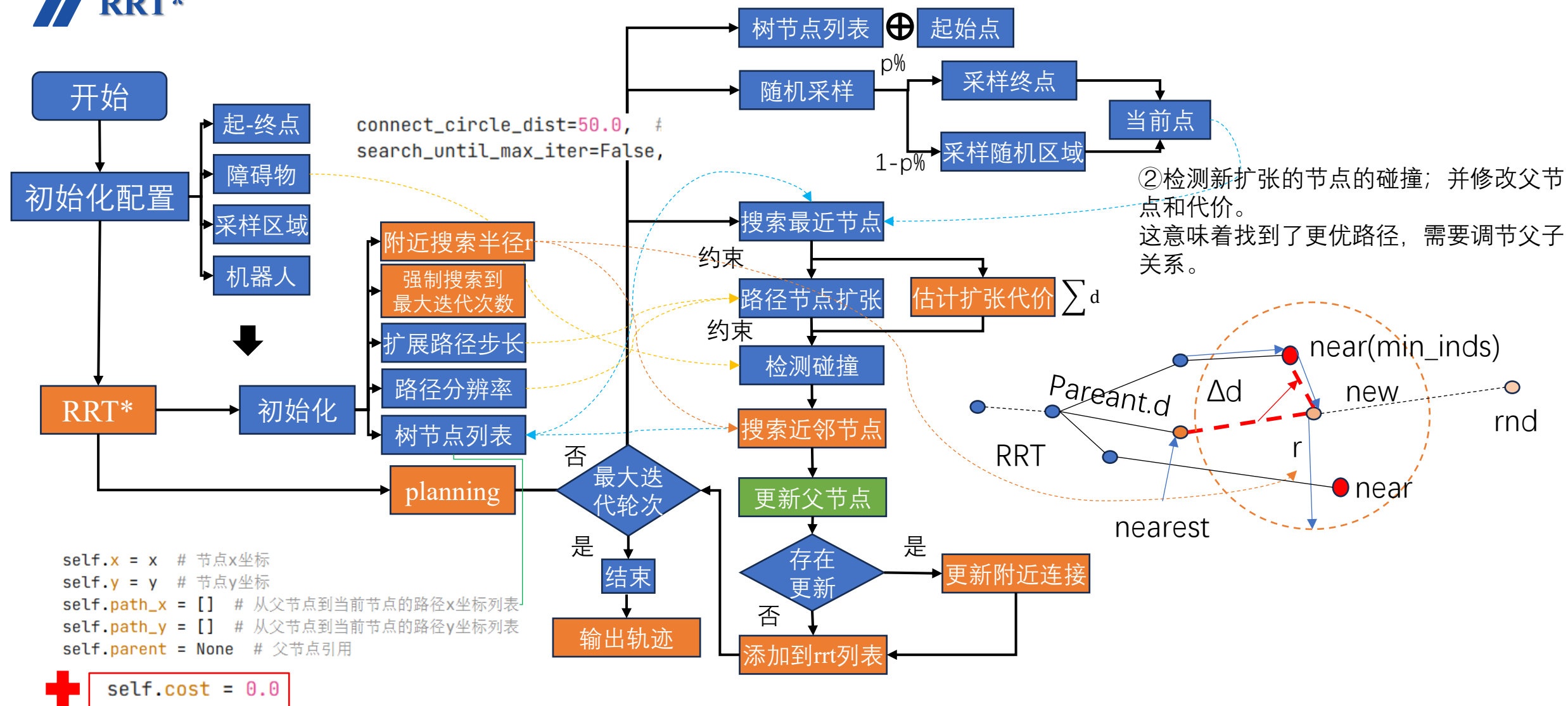


①和A*算法利用启发式距离类似：对当前节点半径r内的其它节点进行扩张，并计算代价。找出其中代价最小的替代nearest节点，构建父子关系。



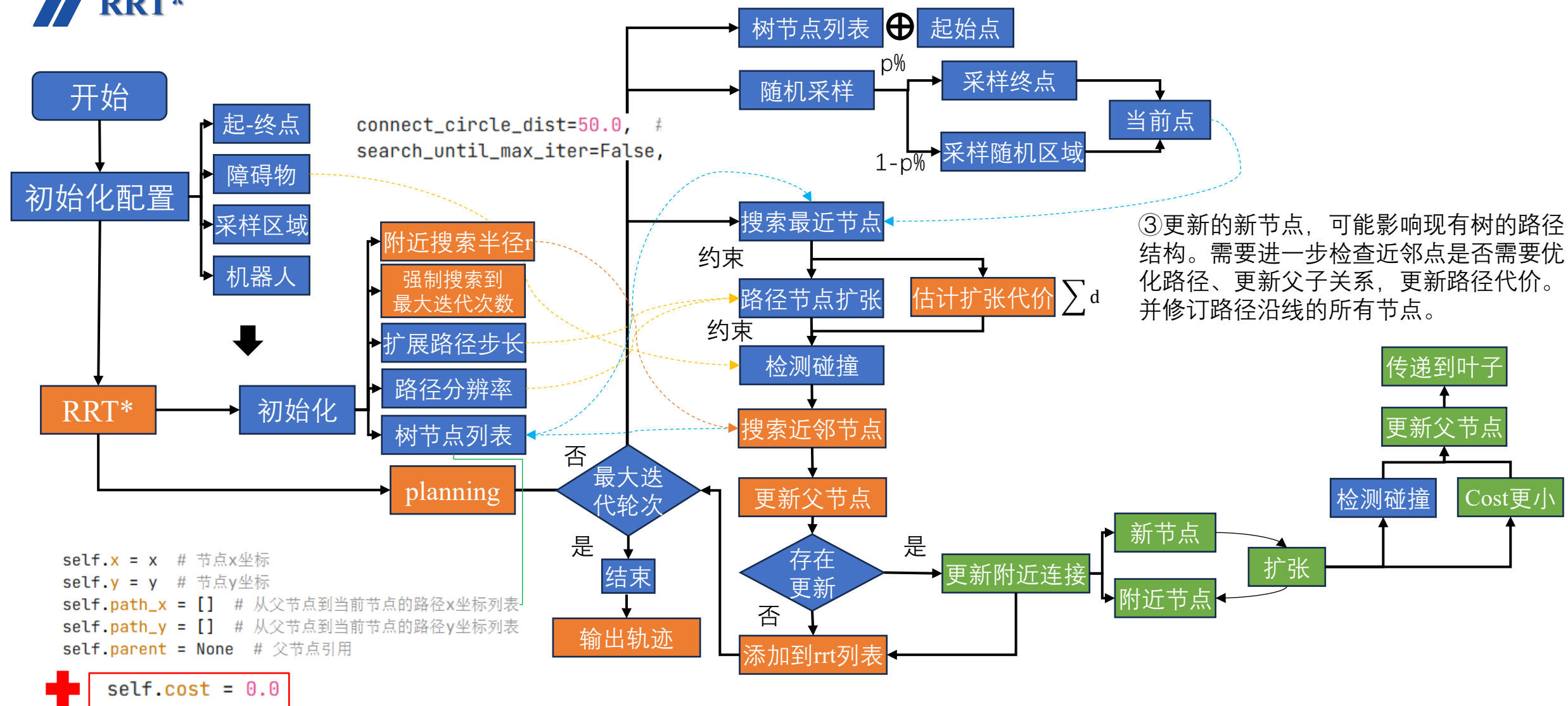
Part 4 | 随机树快速搜索 star

» RRT*



Part 4 | 随机树快速搜索 star

» RRT*



目录

Contents

01 RRT

02 RRT with path smoothing

03 RRT with sobol sampler

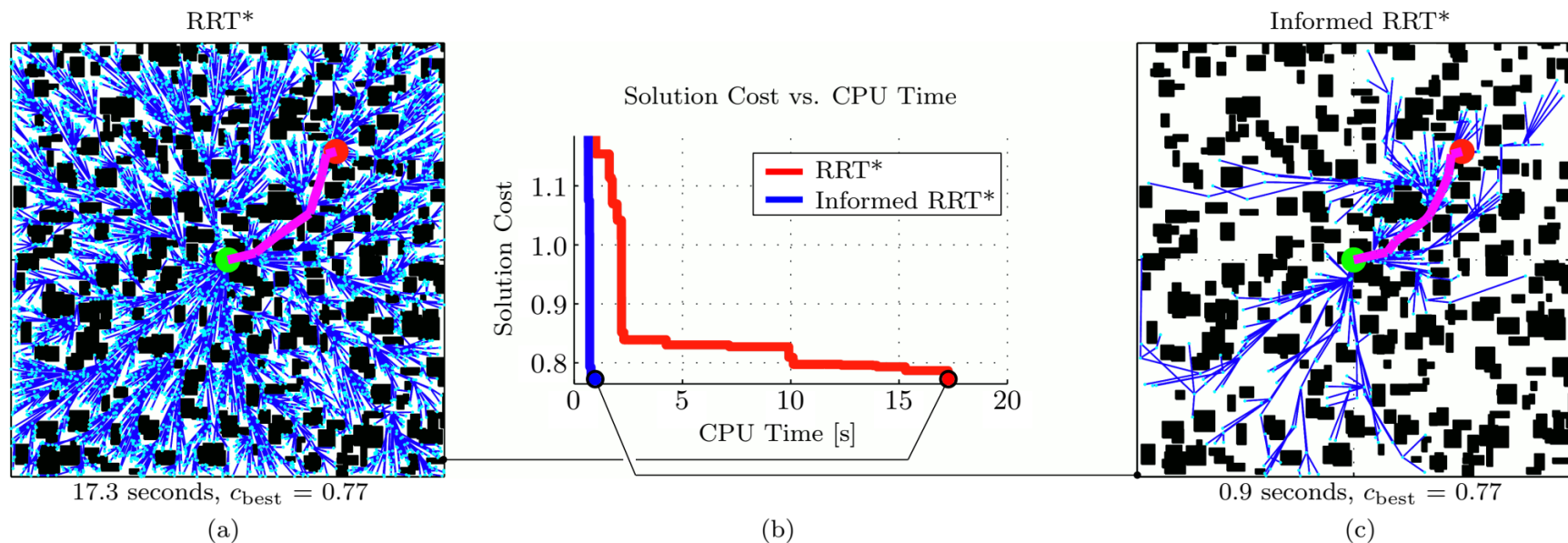
04 RRT star

05 Informed RRT star

Part 5 | Informed RRT star

» Informed RRT*

[论文链接](#)



RRT star与informed RRT star的对比，无论是总时间和总搜索代价（cost）均要优于原算法。

量化评估体系：

时空复杂度， $O(n)$ 是衡量通用算法的两个指标。在特定算法上，基于任务本身设计针对性的量化评估指标，从不同角度衡量算法的改进情况，是算法优化的基础。

CPU time：单位为秒，可以认为是在同等硬件条件下，相同线程测试的运行时间。

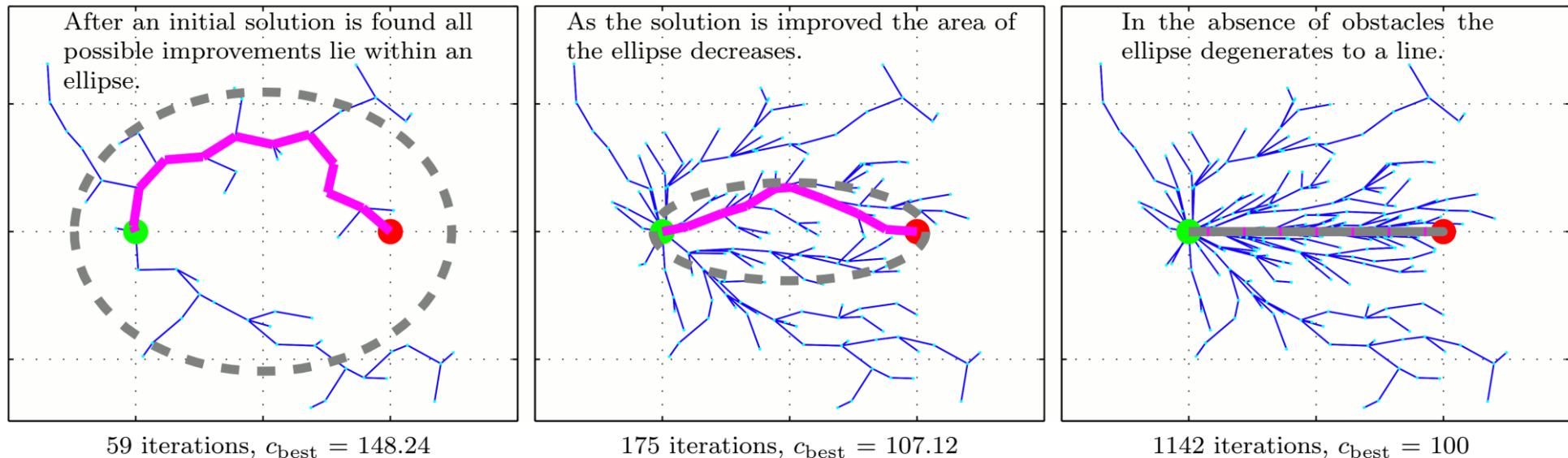
Solution cost：可以认为是搜索路径的总和。

通过上述两个维度，可以简单对比算法的效率。

Part 5 | Informed RRT star

» Informed RRT*

椭圆是基于路径生成并优化的：只有找到从起点-终点的一条可行路径，才能进行椭圆优化。

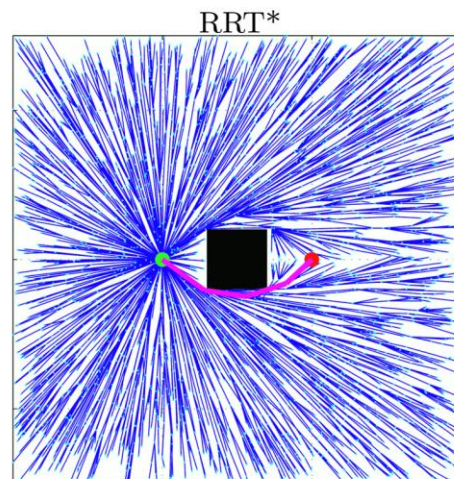
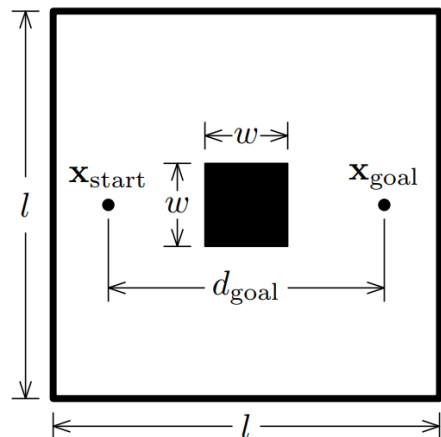


只有特定子集的状态^(a)能帮助改进当前路径（比如让路径更短），这个子集^(c)的几何形状可以描述为“长轴超球面（prolate hyperspheroid）”——可以理解为一个“沿着当前最优路径方向拉长的高维球体”，只有落在这个“球”里的状态，才有可能优化现有路径。在2D平面，这个椭球的投影是椭圆。

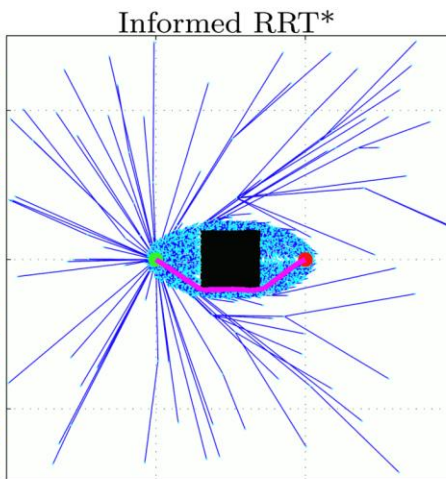
同时，通过迭代子集，不断基于最新路径，压缩搜索空间，可以逼近最优（上图为没有障碍物的理想情况）。

Part 5 | Informed RRT star

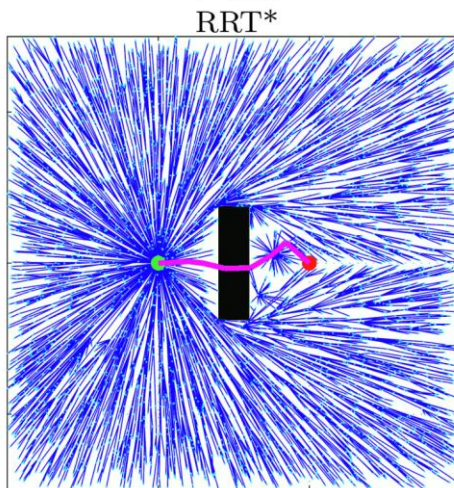
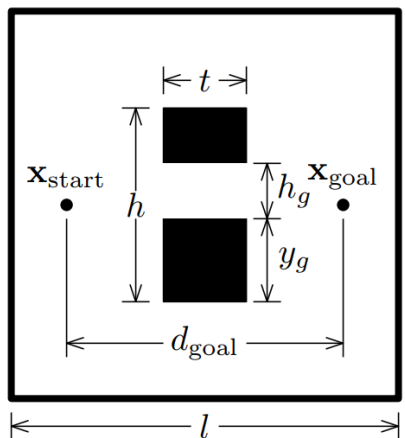
» Informed RRT*



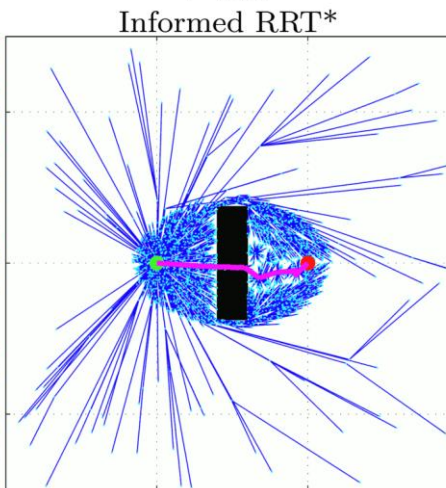
5 seconds, $c_{best} = 114.86$



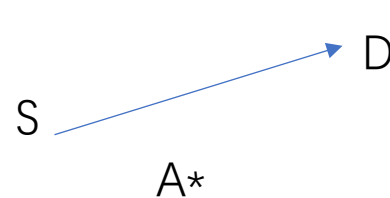
5 seconds, $c_{best} = 112.52$



12.32 seconds



4.00 seconds



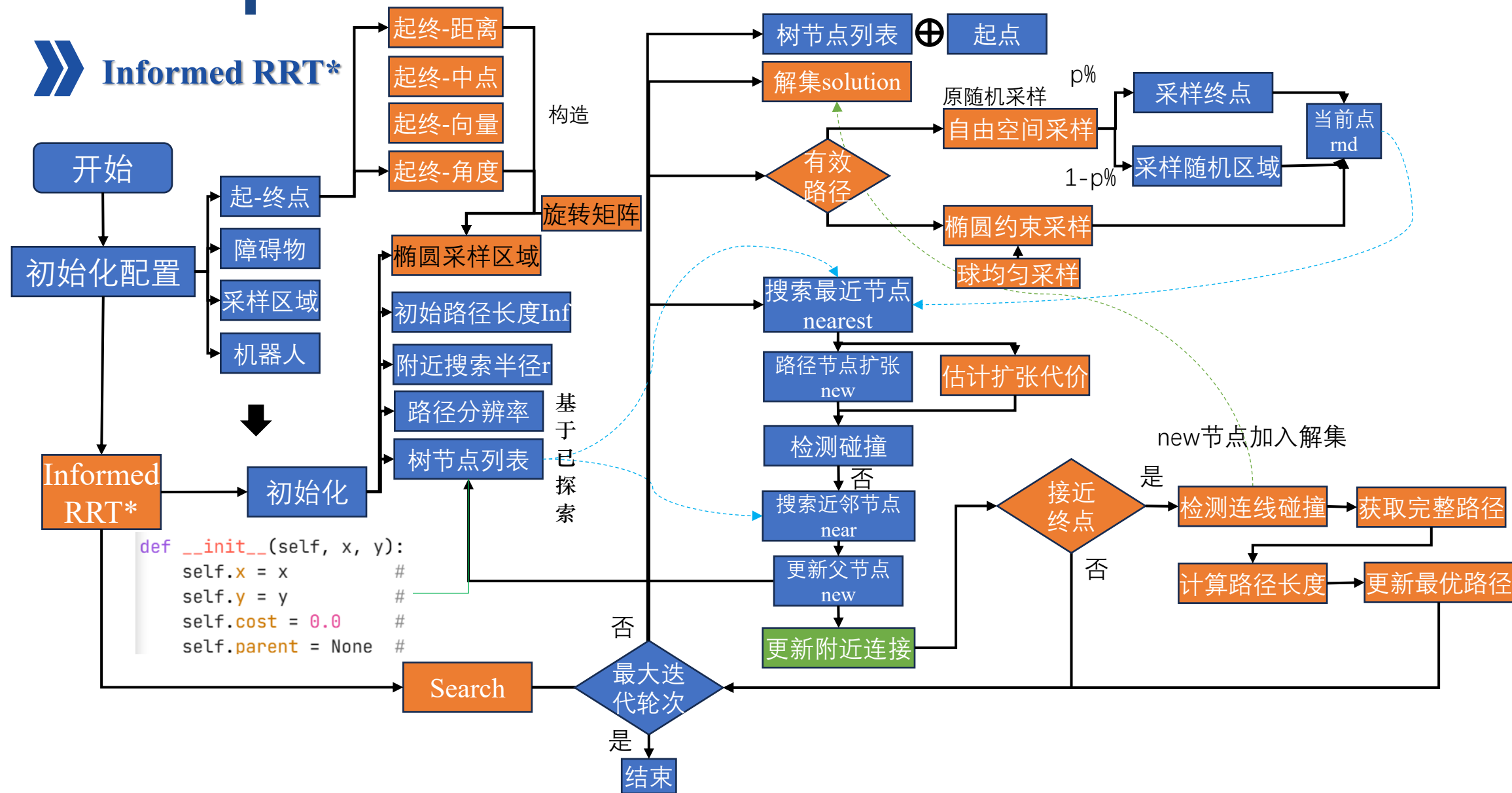
在此情形中，障碍物为搜索空间提供了一个下界，障碍物的宽度（正方形）是影响解搜索范围的因素。最终，两个算法的解近似相同，但是搜索代价后者要小得多。

在此情形中，障碍物中间存在一个通路，通路的宽度是影响搜索的关键因素。同样，虽然两个算法都找到了穿过中间的路径，但RRT Star付出的搜索代价巨大。

Informed RRT*可以看作是基于 长轴超球面 启发搜索的路径规划算法。

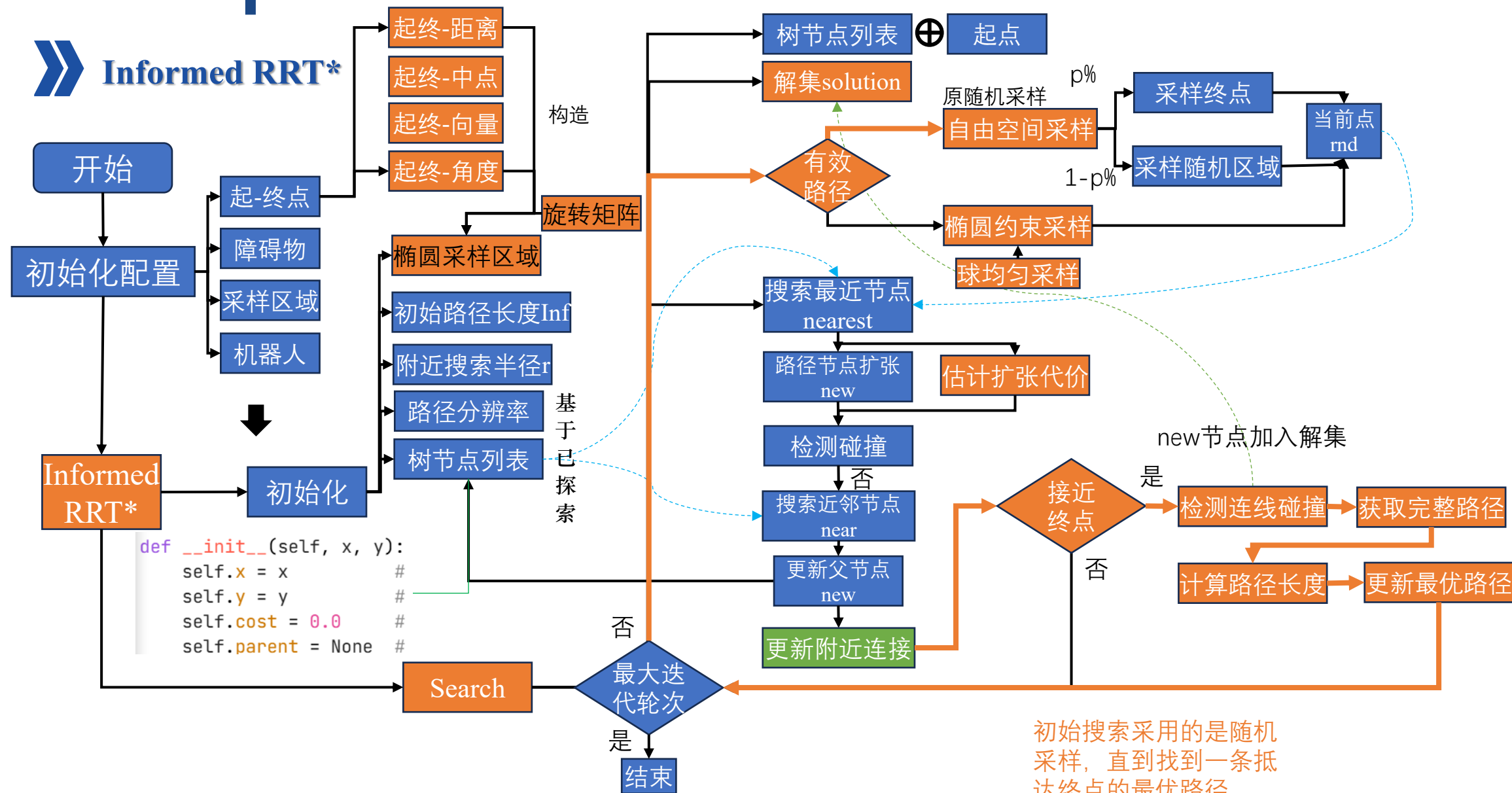
Part 5 | Informed RRT star

» Informed RRT*



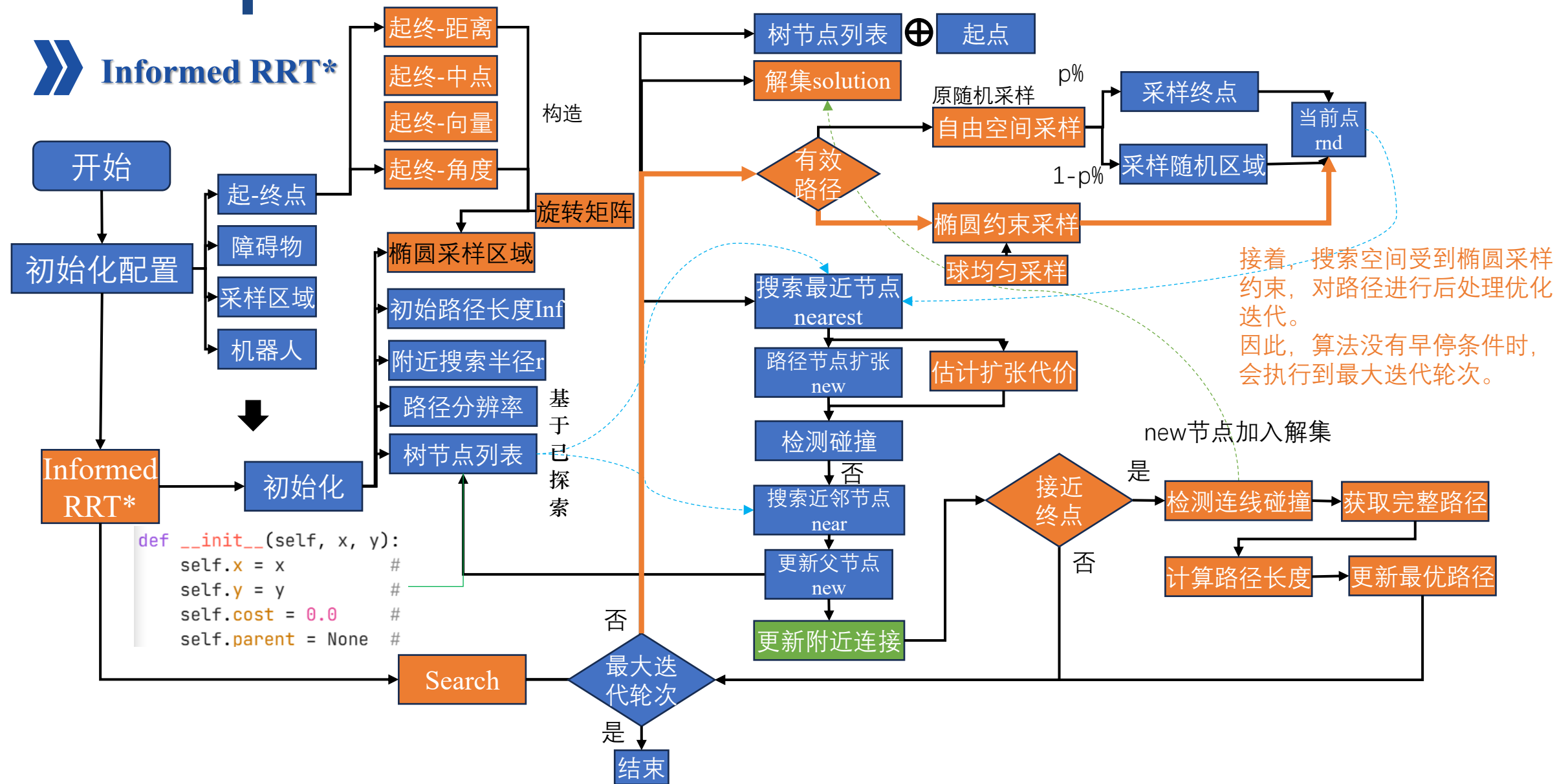
Part 5 | Informed RRT star

» Informed RRT*



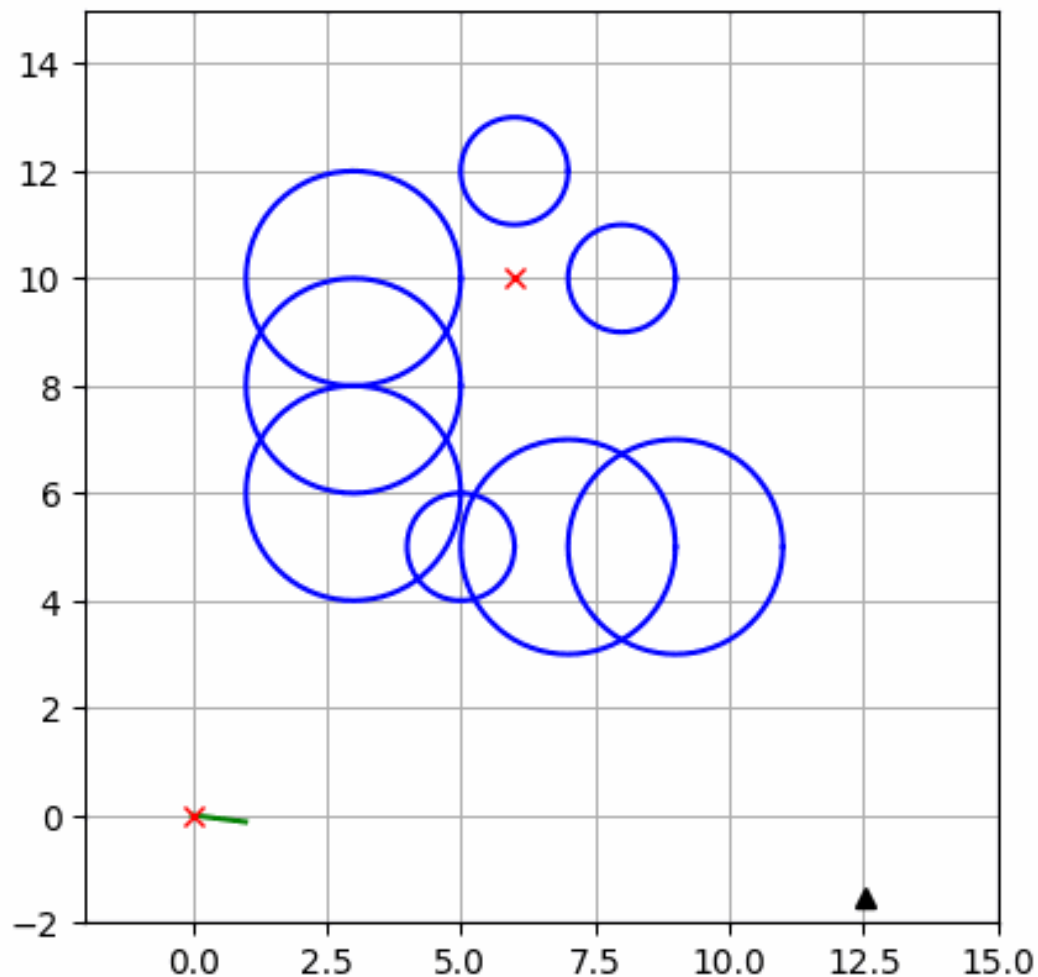
Part 5 | Informed RRT star

» Informed RRT*



Part 5 | Informed RRT star

» Informed RRT*



实验要求: 1) 阅读Informed RRT*论文，了解基于CPU time和Solution cost的量化评估体系，实现相应的计算函数。

2) 对障碍物数量、大小进行随机化生成。

3) 将本章节所介绍的RRT及其变种算法进行量化评估对比，指标采用1) 中实现的函数。量化要求至少进行十种随机场景下的十次重复算法执行。

4) 保持其他参数一致的情况下，对结果进行直方图+误差bar的形式进行可视化绘制。

开始实验



北京航空航天大学
BEIHANG UNIVERSITY